

Punto 3.10 — Explicit Loading

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.9. Se mantiene la separación Domain/Application/Infrastructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Explicit Loading** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.10
dotnet restore AceriaData.sln
```

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture.LINQ` y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerConCargaExplicita`, `ObtenerConPlanchasPesadasExplicitas`, `IgnoreAutoIncludes`, `IsLoaded`, `Query().Where`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/CargaExplicitaUseCase.cs`

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class CargaExplicitaUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public CargaExplicitaUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== EXPLICIT LOADING ===");
        var orden = _unidad.Ordenes.ObtenerConCargaExplicita("OF-2024-0001");
        var filtrada = _unidad.Ordenes.ObtenerConPlanchasPesadasExplicitas("OF-2024-0002", 300m);
        if (orden is null || orden.Planchas.Count != 2 || orden.Detalle is null) throw new
        InvalidOperationException("Carga explícita incompleta.");
        if (filtrada is null || filtrada.Planchas.Count != 0) throw new
        InvalidOperationException("Query() de carga explícita inesperado.");
        Console.WriteLine($"Carga completa: {orden.Planchas.Count} planchas | Filtrada OF-0002:
        {filtrada.Planchas.Count}");
    }
}
```

Explicación línea a línea del caso de uso 3.10

Línea 1: `using AceriaData.Application.Interfaces;` → Importa los puertos de Application, en especial IUnidadDeTrabajo e IOrdenRepositorio, que desacoplan el caso de uso de EF Core.

Línea 3: `namespace AceriaData.Application.UseCases;` → Declara el espacio de nombres de la capa a la que pertenece el archivo.

Línea 5: `public sealed class CargaExplicitaUseCase` → Declara el caso de uso concreto que coordina la demostración del punto.

Línea 6: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.

Línea 8: `public CargaExplicitaUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → Recibe la unidad de trabajo mediante inyección de dependencias.

Línea 10: `public void Ejecutar()` → Define la operación docente que ejecutará el composition root.

Línea 11: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 12: `Console.WriteLine("=== EXPLICIT LOADING ===");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 13: `var orden = _unidad.Ordenes.ObtenerConCargaExplicita("OF-2024-0001");` → Carga la orden indicada y, desde Infrastructure, carga explícitamente las navegaciones requeridas antes de devolverla.

Línea 14: `var filtrada = _unidad.Ordenes.ObtenerConPlanchasPesadasExplicitas("OF-2024-0002", 300m);` → Usa Query() sobre la colección para cargar explícitamente solo las planchas que superan el umbral de peso.

Línea 15: `if (orden is null || orden.Planchas.Count != 2 || orden.Detalle is null) throw new InvalidOperationException("Carga explícita incompleta.");` → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.

Línea 16: `if (filtrada is null || filtrada.Planchas.Count != 0) throw new InvalidOperationException("Query() de carga explícita inesperado.");` → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.

Línea 17: `Console.WriteLine($"Carga completa: {orden.Planchas.Count} planchas | Filtrada OF-0002: {filtrada.Planchas.Count}");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 18: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Línea 19: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

▮ Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```

using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<CargaExplicitaUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<CargaExplicitaUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.10 OK");

```

Explicación línea a línea de Program.cs 3.10

Línea 1: `using AceriaData.Application.UseCases;` → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.

Línea 2: `using AceriaData.ConsoleApp;` → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3: `using AceriaData.Infrastructure;` → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4: `using AceriaData.Infrastructure.Persistence;` → Importa AceriaDbContext para preparar la base de demostración desde el composition root.

Línea 5: `using Microsoft.EntityFrameworkCore;` → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.

Línea 6: `using Microsoft.Extensions.Configuration;` → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.

Línea 7: `using Microsoft.Extensions.DependencyInjection;` → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.

Línea 9: `var configuration = new ConfigurationBuilder()` → Inicia la construcción de la configuración de la aplicación.

Línea 10: `.SetBasePath(AppContext.BaseDirectory)` → Fija como base el directorio de ejecución para localizar appsettings.json.

Línea 11: `.AddJsonFile("appsettings.json", optional: false)` → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12: `.AddEnvironmentVariables()` → Permite sobrescribir configuración mediante variables de entorno.

Línea 13: `.Build();` → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer `ConnectionStrings:AceriaDB`.

Línea 15: `var connectionString = configuration.GetConnectionString("AceriaDB")` → Obtiene la cadena `AceriaDB` desde la configuración.

Línea 16: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → Detiene el arranque con un error explícito si no existe la cadena de conexión `AceriaDB`, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18: `var services = new ServiceCollection();` → Crea el contenedor de servicios del `composition root`.

Línea 19: `services.AddAceriaInfrastructure(connectionString);` → Registra `DbContext`, repositorio y unidad de trabajo de `Infrastructure`.

Línea 20: `services.AddScoped<CargaExplicitaUseCase>();` → Registra el caso de uso con ciclo de vida `Scoped` para compartir el mismo ámbito que `DbContext`.

Línea 22: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → Construye el proveedor de servicios y lo declara con `using` para liberar correctamente los servicios `IDisposable` al terminar el proceso.

Línea 23: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24: `ValidateOnBuild = true,` → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.

Línea 25: `ValidateScopes = true` → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio `Scoped` consumido incorrectamente desde un `Singleton`.

Línea 26: `});` → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27: `using var scope = provider.CreateScope();` → Crea el ámbito `Scoped` que compartirá `AceriaDbContext`, `IUnidadDeTrabajo` y el caso de uso durante esta ejecución.

Línea 29: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` para preparar la base de demostración.

Línea 30: `context.Database.EnsureDeleted();` → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.

Línea 31: `context.Database.Migrate();` → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.

Línea 32: `DemoData.Seed(context);` → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.

Línea 33: `context.ChangeTracker.Clear();` → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35: `var useCase = scope.ServiceProvider.GetRequiredService<CargaExplicitaUseCase>();` → Resuelve el caso de uso concreto registrado para este checkpoint.

Línea 36: `useCase.Ejecutar();` → Ejecuta la demostración y sus aserciones E2E.

Línea 38: `Console.WriteLine("3.10 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.10 OK
```

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Cargar explícitamente una navegación que ya estaba cargada puede repetir trabajo; `IsLoaded` permite evitarlo.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Código activable para reproducir el error controlado

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/Interfaces.cs`

Marcador: APOYO M03 3.10 - DIAGNOSTICO ISLOADED

```
// Se activa únicamente en la copia desechable del Paso 9.
string DiagnosticarIsLoaded(string numeroOrden);
```

Archivo: `src/AceriaData.Infrastructure/Repositories/Repositories.cs`

Marcador: APOYO M03 3.10 - DIAGNOSTICO ISLOADED

```
// Cuenta cuántas llamadas a Load() son realmente necesarias sobre la misma navegación.
public string DiagnosticarIsLoaded(string numeroOrden)
{
    var orden = _context.OrdenesFabricacion.IgnoreAutoIncludes()
        .SingleOrDefault(o => o.NumeroOrden == numeroOrden);
    if (orden is null) return "Orden no encontrada";

    var planchas = _context.Entry(orden).Collection(o => o.Planchas);
    var antes = planchas.IsLoaded;
    var cargas = 0;

    if (!planhas.IsLoaded)
    {
        planchas.Load();
        cargas++;
    }

    var despues = planchas.IsLoaded;

    if (!planhas.IsLoaded)
    {
        planchas.Load();
        cargas++;
    }

    return $"Antes: {antes} | Después: {despues} | Loads ejecutados: {cargas} | Planchas:
{orden.Planchas.Count}";
}
```

Archivo: src/AceriaData.Application/CargaExplicitaUseCase.cs

Marcador: ERROR CONTROLADO M03 3.10 - EVITAR CARGA REPETIDA CON ISLOADED

```
// Demuestra el Paso 9: IsLoaded permite no ejecutar Load() una segunda vez.
var diagnostico = _unidad.Ordenes.DiagnosticarIsLoaded("0F-2024-0003");
const string esperado = "Antes: False | Después: True | Loads ejecutados: 1 | Planchas: 1";
if (diagnostico != esperado)
    throw new InvalidOperationException($"Error controlado 3.10 inesperado: {diagnostico}");
Console.WriteLine($"Error controlado 3.10 OK | {diagnostico}");
```

Comprobación esperada: IsLoaded evita repetir una carga explícita ya completada.

Laboratorio adicional del punto 3.10

Diagnóstico técnico

Explicit Loading parte de una entidad rastreada y decide cuándo ejecutar `Collection().Load()` o `Reference().Load()`. `IsLoaded` evita repetir una carga; `Query()` permite filtrar antes de `Load()`. Las consultas raíz neutralizan `AutoInclude` para no falsear la demostración.

Reto resuelto y verificación adicional

Carga 0F-2024-0001: espera dos planchas y detalle. Después carga las planchas de 0F-2024-0002 con peso mínimo 300 kg: espera colección vacía. Así se demuestra que `Query()` altera la consulta de navegación antes de materializar.

Errores comunes revisados

Error	Causa	Solución
Estado de carga no entendido	Se desconoce si la navegación está completa	Consultar <code>IsLoaded</code> para explicitar el estado; si ya es <code>true</code> , <code>Load()</code> es un no-op
Filtro no aplicado	Se aplicó el filtro fuera del Query	Aplicar el filtro dentro del Query

Error	Causa	Solución
NullReferenceException	La entidad principal es null	Comprobar antes de llamar a Load
Referencia circular	Se carga una entidad que referencia a la principal	Usar DTOs o evitar cargar la referencia inversa
Muchas consultas explícitas	Se llama a Load() para cada entidad dentro de un bucle	Si todas las relaciones se necesitan, valorar Eager Loading o una proyección; si la carga es condicional, medir el coste
Estrategia de carga difícil de razonar	Se mezclan Lazy y Explicit sin intención documentada	Definir qué navegación usa cada estrategia y evitar cargas implícitas inesperadas

Analogía operativa

La carga Explicit en una acería es como pedirle al archivo central que traiga una carpeta relacionada solo cuando se decide que hace falta. `IsLoaded` es como comprobar si ya se dispone de esa carpeta antes de pedirla de nuevo y `Query()` permite pedir solo los documentos que cumplen una condición. Si las relaciones necesarias se conocen desde el principio, Eager Loading o una proyección pueden reducir viajes al archivo; si la necesidad aparece de forma condicional o se quiere filtrar una navegación concreta, Explicit Loading puede resultar más apropiada. No existe una opción universalmente más eficiente: se comparan número de consultas, volumen transferido y claridad del flujo.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Carga únicamente planchas por encima de un peso mínimo mediante `Collection(...).Query()`.

Cómo activar el reto en el checkpoint

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/CargaExplicitaUseCase.cs`

Marcador: RETO M03 3.10 - QUERY FILTRADA POR PESO MINIMO

```
// Demuestra el Paso 10 con otra orden y confirma que el filtro se aplica antes de Load().
var reto = _unidad Ordenes.ObtenerConPlanchasPesadasExplicitas("OF-2024-0005", 150m);
if (reto is null || reto.Planchas.Count != 1)
    throw new InvalidOperationException("Reto 3.10: se esperaba una única plancha >= 150 kg en OF-2024-0005.");
if (filtrada is null || filtrada.Planchas.Count != 0)
    throw new InvalidOperationException("Reto 3.10: OF-2024-0002 no debe tener planchas >= 300 kg.");
Console.WriteLine("Reto 3.10 OK | OF-2024-0005 >=150kg: 1 | OF-2024-0002 >=300kg: 0");
```

Comprobación esperada: `Collection(...).Query()` filtra la navegación por peso antes de materializar.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.10 OK, el siguiente estado parte exactamente de esta solución y añade

Composición de consultas y ejecución diferida.